What are Microservices? Decomposition, Governance & Trade-offs

Decomposition of Monolithic Ceilings, Conway's Law, Blast Radii, and Team Autonomy

EXECUTIVE SUMMARY

Microservices structure software as a suite of small, independently deployable services modeled around business capabilities. Rather than an aesthetic goal, microservices are an organizational and operational mechanism designed to resolve deployment contention and organizational coordination bottlenecks as engineering headcount scales past 50–100 developers.

1. MONOLITH VS. MICROSERVICE ARCHITECTURE



2. THE THREE MONOLITHIC CEILINGS

1. Deploy Pipeline Lock

Dozens of PRs merged daily cause constant broken master builds. Release trains stall, delaying critical features.

2. Hardware Inefficiency

A single CPU-heavy image transcoder forces the entire monolith cluster to scale on expensive high-CPU VM tiers.

3. Blast Radius Contagion

A fatal null pointer or OOM crash in marketing reporting brings down the primary checkout engine.

3. THE 6 ARCHITECTURAL PILLARS OF MICROSERVICES

Pillar	Operating Principle	Organizational Impact
Independent Deployability	A service can deploy to production at any time without coordinating with other teams.	CI/CD cycle time drops from weeks to minutes.
Business Boundary Focus	Organized along DDD Bounded Contexts (Orders, Billing, Shipping) rather than technical layers.	Eliminates handoffs between DBAs and developers.
Decentralized Persistence	Each service strictly owns its private database. No direct cross-table SQL joins allowed.	Enables schema freedom and polyglot engines.
Smart Endpoints / Dumb Pipes	Business logic resides in application code; the network fabric simply delivers raw bytes.	Avoids brittle centralized Enterprise Service Buses.

4. CONWAY'S LAW & THE TWO-PIZZA TEAM PARADIGM

Melvin Conway established that software architecture mirrors organization communication:

```
Conway's Law Formulation:
  Communication Paths = N * (N - 1) / 2
  * 10 Engineers  --> 45 communication paths (Manageable in monolith)
  * 100 Engineers --> 4,950 communication paths! (Gridlock & coordination fatigue)

Inverse Conway Maneuver:
  Decompose teams into autonomous 'Two-Pizza Teams' (6-8 engineers).
  Assign each team 100% end-to-end ownership of 1-3 bounded microservices.
  Inter-team communication is formalized into versioned API contracts.
```

5. THE ECONOMIC DECISION: WHEN TO BUILD MICROSERVICES

Remain with Monolith When:

- Early startup stage searching for product-market fit.
- Engineering organization has < 30 developers.
- Core domain boundaries and data schemas are rapidly mutating.
- Zero dedicated platform / DevOps engineering support.

Transition to Microservices When:

- Release coordination is the #1 company bottleneck.
- Engineering headcount exceeds 50–100 engineers.
- Specific components require independent, heterogeneous scaling.
- Domain boundaries and team ownership lines have stabilized.

6. THE STRANGLER FIG DECOMPOSITION SEQUENCE

```
// Incremental Monolith Decomposition Protocol:
1. Intercept: Deploy Envoy Edge Proxy in front of Monolith.
2. Carve: Build new Payment Microservice with private PostgreSQL instance.
3. Shift: Route 1% of traffic to new service; verify Prometheus error rates.
4. Scale: Shift 100% traffic to microservice; deprecate legacy payment module in monolith.
5. Repeat for Inventory, Shipping, and Catalog domains.
```

7. PRODUCTION ENGINEERING VERIFICATION CHECKLIST

Foundational Readiness Gates

1. **Modularize First:** Enforce clean modular packages inside the monolith before carving out network services.
2. **Autonomous Deployment Test:** Verify Service A can release to production while Service B is mid-deploy.
3. **Standard Observability:** Every service must export OpenTelemetry traces, Prometheus metrics, and JSON logs.
4. **Two-Pizza Team Staffing:** Never assign 1 developer to 5 microservices; ensure sustainable team ownership.